

Hệ Thống Kiểm Soát Ra Vào Bằng Nhận Diện Khuôn Mặt

DeepFace Access Control

Báo cáo phân tích, thiết kế, triển khai và đánh giá hệ thống AI end-to-end

Dự án	DeepFace Access Control
Môn học	Thực hành phát triển hệ thống trí tuệ nhân tạo
Use case	Xác minh nhân viên tại cổng ra vào, ghi access log và giám sát vận hành.
Phạm vi	Frontend, Backend API, Worker AI, PostgreSQL, Redis, MinIO, Qdrant, Nginx, Prometheus/Grafana, CI/CD baseline.
Tác giả	Nhóm phát triển hệ thống
Link Github	https://github.com/trunguet/2526-THPTHTTTNT-DeepFace

Thành phần hệ thống

Frontend, API, AI worker, dữ liệu, vector search và monitoring trong một stack Docker Compose

React Frontend

FastAPI Backend

DeepFace Worker

PostgreSQL / MinIO / Qdrant

Prometheus / Grafana

Docker Compose

Mục lục

Tóm tắt	1
1 Giới thiệu	2
1.1 Bối cảnh	2
1.2 Use case lựa chọn	2
1.3 Mục tiêu	2
1.4 Phạm vi	3
2 Đối chiếu yêu cầu đề bài	4
2.1 Checklist bắt buộc	4
2.2 Điểm cộng đã chuẩn bị	5
3 Thiết kế kiến trúc	6
3.1 Nguyên tắc thiết kế	6
3.2 Kiến trúc tổng thể	6
3.3 Vai trò từng service	8
3.4 Bảng chứng triển khai trong repo	9
3.5 Lý do dùng xử lý bất đồng bộ	9
4 Thiết kế dữ liệu	11
4.1 PostgreSQL là source of truth	11
4.2 Quan hệ dữ liệu	12
4.3 MinIO, Qdrant và Redis	12
4.4 Trạng thái quan trọng	13
5 Pipeline AI nhận diện khuôn mặt	14
5.1 Cấu hình AI	14
5.2 Luồng đăng ký khuôn mặt	14
5.3 Luồng kiểm tra ra vào	15
5.4 Quy tắc xử lý lỗi	16
6 Backend API	17
6.1 Nhóm endpoint chính	17
6.2 Auth và phân quyền	18

6.3	Validate upload ảnh	18
7	Thiết kế giao diện	19
7.1	Home Gateway	19
7.2	User Terminal	19
7.3	Admin Console	20
8	Monitoring và vận hành	22
8.1	Mục tiêu monitoring	22
8.2	Grafana dashboard	23
8.3	Alertmanager	23
9	Triển khai Docker, CI/CD và backup	25
9.1	Docker Compose	25
9.2	CI/CD baseline	26
9.3	Helm baseline	26
9.4	Backup	26
10	Kiểm thử và đánh giá	27
10.1	Chiến lược kiểm thử	27
10.2	Đánh giá MVP	27
11	Bảo mật và rủi ro	29
11.1	Bảo mật hiện tại	29
11.2	Rủi ro cần hardening	29
11.3	Dữ liệu sinh trắc học	29
12	Phân công nhóm và kế hoạch bảo vệ	30
12.1	Phân công nhóm 4 người	30
12.2	Kịch bản demo đề xuất	30
13	Kết luận và hướng phát triển	31
13.1	Kết luận	31
13.2	Hướng phát triển	31
13.3	Đánh giá tổng thể	31
	Tài liệu tham khảo	32

Tóm tắt

Tổng quan điều hành

Báo cáo trình bày hệ thống kiểm soát ra vào bằng nhận diện khuôn mặt theo hướng full-stack. Dự án không chỉ là một script nhận diện ảnh, mà là một hệ thống có giao diện người dùng, giao diện quản trị, API nghiệp vụ, worker xử lý AI nền, PostgreSQL, Redis, MinIO, Qdrant, Nginx và monitoring.

processing

embedding

vector search

granted

denied / error

3 UI

Home Gateway, User Terminal và Admin Console tách rõ theo vai trò.

API + AI

FastAPI điều phối nghiệp vụ, DeepFace worker xử lý ảnh nền.

Full stack

PostgreSQL, Redis, MinIO, Qdrant, Nginx, Prometheus và Grafana.

Đóng góp chính của MVP

Trải nghiệm người dùng

Frontend

User Terminal hỗ trợ gửi snapshot/webcam, xem trạng thái xử lý và lịch sử truy cập.

Quản trị hệ thống

Admin

Admin Console quản lý employees, users, cameras, enrollment và access logs.

Pipeline AI

Worker

DeepFace tạo embedding, Qdrant tìm vector gần nhất và worker cập nhật quyết định truy cập.

Hạ tầng demo

Ops

Docker Compose đóng gói backend, frontend, database, queue, storage, vector DB và monitoring.

Kiểm chứng chất lượng

Testing

Backend tests, worker tests, frontend build và script baseline giúp kiểm tra nhanh trạng thái hệ thống local.

Giới thiệu

1.1 Bối cảnh

Các hệ thống kiểm soát ra vào truyền thống thường dùng thẻ từ, mã PIN hoặc xác nhận thủ công. Cách tiếp cận này đơn giản nhưng tồn tại nhiều hạn chế: thẻ có thể bị mượn hộ, mã PIN có thể bị chia sẻ, còn kiểm tra thủ công phụ thuộc vào con người và khó ghi nhận lịch sử một cách nhất quán.

Nhận diện khuôn mặt là một hướng phù hợp cho bài toán xác minh nhanh tại cổng ra vào. Người dùng chỉ cần đứng trước camera hoặc gửi snapshot, hệ thống sẽ phát hiện khuôn mặt, tạo vector embedding, so khớp với dữ liệu đã đăng ký và ghi lại quyết định truy cập.

1.2 Use case lựa chọn

Use case của nhóm là **kiểm soát ra vào công ty bằng nhận diện khuôn mặt**. Admin đăng ký nhân viên, upload ảnh khuôn mặt mẫu và theo dõi log. User hoặc nhân viên tại terminal gửi ảnh kiểm tra quyền. Hệ thống trả về một trong các trạng thái:

- ▶ **processing**: backend đã nhận request và worker đang chờ xử lý.
- ▶ **granted**: ảnh khớp với một nhân viên active và vượt ngưỡng matching.
- ▶ **denied**: có xử lý nhưng không đủ điều kiện cấp quyền.
- ▶ **error**: ảnh lỗi, không có mặt, nhiều mặt hoặc pipeline gặp lỗi dependency.

1.3 Mục tiêu

Mục tiêu của đề án là xây dựng một MVP đủ các thành phần của một hệ thống AI engineering end-to-end:

Mục tiêu cụ thể

1. Có frontend người dùng để gửi ảnh kiểm tra quyền ra vào và xem kết quả.
2. Có frontend admin để quản lý nhân viên, tài khoản, camera, ảnh mẫu và log.
3. Có backend API để xác thực, phân quyền, CRUD, upload ảnh và queue job.
4. Có worker AI xử lý tác vụ nặng ở background thay vì chặn request HTTP.
5. Có PostgreSQL, MinIO, Qdrant và Redis đúng vai trò lưu trữ/queue.
6. Có Nginx làm gateway, Docker Compose để chạy hệ thống bằng một lệnh.
7. Có monitoring bằng Prometheus/Grafana và baseline CI/CD.

1.4 Phạm vi

Báo cáo tập trung vào phân tích yêu cầu, kiến trúc, thiết kế dữ liệu, pipeline AI, API, giao diện, triển khai Docker, monitoring, backup, kiểm thử, bảo mật và hướng phát triển. Các phần ngoài phạm vi MVP gồm triển khai camera vật lý tại hiện trường, đánh giá accuracy trên dataset lớn, hardening production đầy đủ và auto-deploy lên một cluster Kubernetes thật.

Đổi chiều yêu cầu đề bài

2.1 Checklist bắt buộc

Bảng 2.1: Mức độ đáp ứng yêu cầu bài tập lớn

Yêu cầu	Cách hệ thống đáp ứng	Trạng thái
Use case rõ ràng	Kiểm soát ra vào công ty/phòng lab bằng nhận diện khuôn mặt nhân viên.	Đạt
Frontend người dùng	User Terminal gửi snapshot/webcam, hiển thị trạng thái và lịch sử access.	Đạt
Frontend admin	Admin Console quản lý employees, users, cameras, enrollment và access logs.	Đạt
Backend API	FastAPI cung cấp auth, employees, cameras, access, logs, admin, health và metrics.	Đạt
Database	PostgreSQL lưu users, employees, cameras, face embeddings metadata và access logs.	Đạt
Object storage	MinIO lưu ảnh đăng ký nhân viên, ảnh snapshot access và có thể lưu backup.	Đạt
Vector database	Qdrant lưu face vectors và tìm embedding gần nhất bằng cosine similarity.	Đạt
Queue / worker nền	Redis có queue <code>embedding_jobs</code> và <code>access_jobs</code> ; worker xử lý DeepFace.	Đạt
Load balancer / gateway	Nginx gom Home, User, Admin và Backend về port 8080.	Đạt
Docker Compose	<code>docker compose up -build -d</code> dựng stack local.	Đạt
README / tài liệu	Repo có docs theo architecture, API, AI pipeline, monitoring, backup, CI/CD và báo cáo này.	Đạt
Không commit secret	Dùng <code>.env.example</code> , biến môi trường và credential demo cần đổi khi production.	Đạt

2.2 Điểm cộng đã chuẩn bị

Bảng 2.2: Các hạng mục mở rộng có thể trình bày khi bảo vệ

Hạng mục	Nội dung trong repo
CI/CD baseline	GitHub Actions chạy backend tests, worker tests, frontend builds, Docker image builds và publish image khi push main.
Kubernetes/Helm	Helm chart baseline cho backend, worker, frontend, database, Redis, MinIO, Qdrant và ingress.
Monitoring	Prometheus scrape backend metrics, Grafana dashboard, Alertmanager local receiver.
Backup	Script backup local/S3 và service cron-backup cho PostgreSQL dump lên MinIO/S3.
Docker Hub readiness	Script kiểm tra image namespace/tag giữa Compose, Helm và Docker Hub.

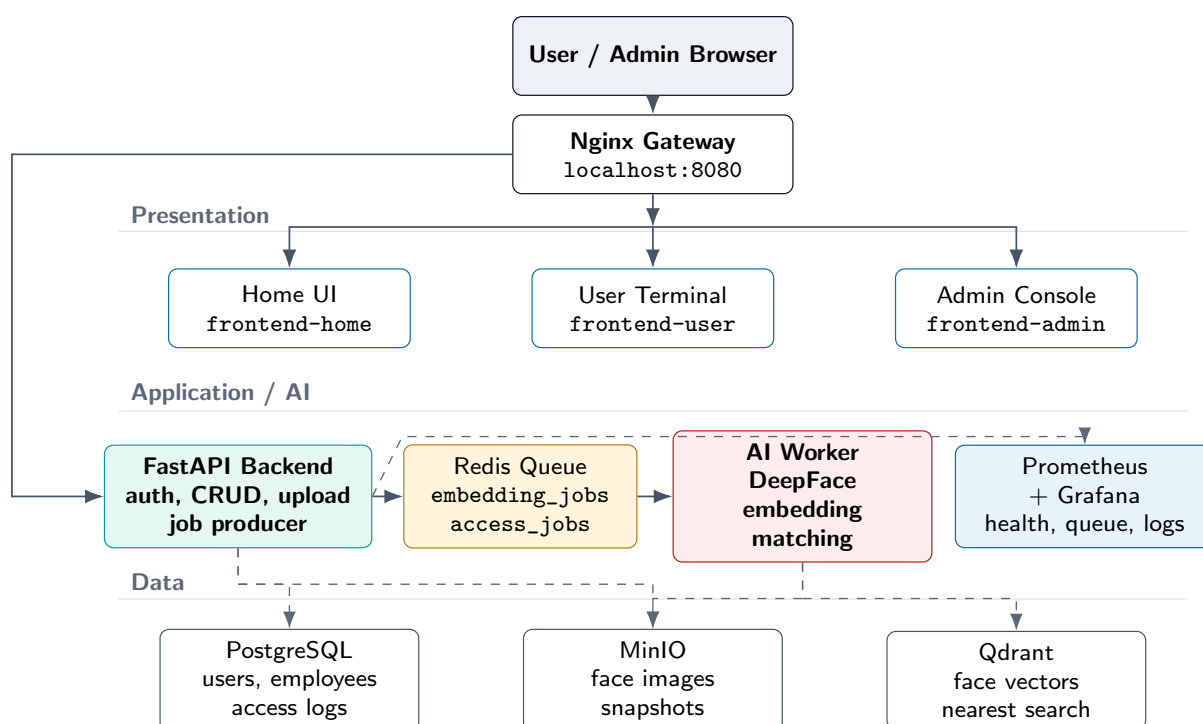
Thiết kế kiến trúc

3.1 Nguyên tắc thiết kế

Hệ thống được thiết kế theo hướng tách service. Mỗi service có một trách nhiệm rõ ràng, giúp dễ kiểm thử, dễ thay thế và dễ scale:

- ▶ Frontend chỉ xử lý trải nghiệm người dùng và gọi API.
- ▶ Backend xác thực, validate, lưu metadata, upload ảnh và queue job.
- ▶ Worker xử lý AI nặng, gồm phát hiện mặt, embedding và matching.
- ▶ PostgreSQL là source of truth cho dữ liệu nghiệp vụ.
- ▶ MinIO lưu file ảnh gốc/snapshot thay vì nhét file lớn vào database.
- ▶ Qdrant là vector database phục vụ tìm kiếm gần nhất.
- ▶ Redis tách request HTTP khỏi xử lý AI nền.
- ▶ Prometheus/Grafana giúp chứng minh hệ thống có khả năng quan sát vận hành.

3.2 Kiến trúc tổng thể



Hình 3.1: Kiến trúc tổng thể của hệ thống DeepFace Access Control

Cách đọc sơ đồ kiến trúc

- ▶ Người dùng chỉ truy cập một cổng duy nhất là `localhost:8080`; Nginx định tuyến tới Home, User Terminal, Admin Console hoặc Backend API.
- ▶ Backend chịu trách nhiệm xác thực, phân quyền, validate ảnh, lưu metadata, upload ảnh vào MinIO và tạo job trong Redis.
- ▶ Redis tách request HTTP khỏi tác vụ AI nặng; backend trả về trạng thái **processing**, còn worker cập nhật kết quả cuối.
- ▶ Worker tải ảnh từ MinIO, phát hiện đúng một khuôn mặt, tạo embedding bằng DeepFace, tìm vector gần nhất trong Qdrant và cập nhật access log trong PostgreSQL.
- ▶ Prometheus/Grafana đọc `/metrics` để theo dõi health, queue length và số lượng log theo trạng thái.

Bảng 3.1: Luồng dữ liệu chính trong kiến trúc

Luồng	Đường đi	Ý nghĩa kỹ thuật
Routing giao diện	Browser → Nginx → Frontend	Tách Home/User/Admin thành các frontend riêng nhưng vẫn dùng một endpoint.
Upload enrollment	Admin UI → Backend → MinIO + Redis	Ảnh mẫu được lưu ở object storage, job embedding chạy nền.
Check access	User UI → Backend → access log + Redis	API trả nhanh bằng trạng thái processing ; worker xử lý sau.
AI matching	Worker → MinIO/Qdrant/PostgreSQL	Worker đọc ảnh, tạo vector, search Qdrant và verify employee active trong PostgreSQL.
Monitoring	Backend <code>/metrics</code> → Prometheus → Grafana	Theo dõi trạng thái runtime, backlog queue và kết quả truy cập.

Ranh giới trách nhiệm giữa các layer

- ▶ **Presentation Layer:** chỉ giữ trạng thái UI, token đăng nhập, form upload và hiển thị kết quả; không xử lý nhận diện khuôn mặt.
- ▶ **Backend/API Layer:** là lớp điều phối nghiệp vụ. Backend kiểm tra quyền, validate request, tạo bản ghi ban đầu và phát sinh job, nhưng không chạy DeepFace trong request path.
- ▶ **AI Worker Layer:** là nơi duy nhất chạy tác vụ AI nặng. Worker có thể scale ngang mà không cần thay đổi frontend hoặc backend.
- ▶ **Data Layer:** PostgreSQL giữ dữ liệu chuẩn, MinIO giữ file ảnh, Qdrant giữ index vector; mỗi hệ thống có vai trò riêng để tránh trộn trách nhiệm.
- ▶ **Observability Layer:** health/metrics/log giúp chứng minh hệ thống vận hành được, không chỉ chạy một luồng demo đơn lẻ.

Bảng 3.2: Ánh xạ yêu cầu hệ thống sang quyết định thiết kế

Yêu cầu	Quyết định thiết kế	Lý do và bằng chứng triển khai
Tách user/admin	Ba frontend riêng: Home, User Terminal, Admin Console; Nginx gom về một entypoint.	Người dùng thao tác access check ở terminal, admin quản lý employees/cameras/logs ở console. Cách này giảm nhầm lẫn quyền và dễ demo theo vai trò.
API phản hồi nhanh	Backend chỉ tạo access log trạng thái processing và đẩy job vào Redis.	Request HTTP không bị chặn bởi DeepFace; trạng thái cuối được worker cập nhật sau. Đây là thiết kế phù hợp với tác vụ AI có latency biến động.
Lưu ảnh an toàn hơn	Ảnh gốc và snapshot lưu ở MinIO, database chỉ lưu metadata/object key.	PostgreSQL không phải chứa file nhị phân lớn; sau này có thể thêm retention policy, backup bucket hoặc object lifecycle.
So khớp khuôn mặt nhanh	Embedding lưu/index trong Qdrant, truy vấn nearest vector khi có snapshot mới.	Vector search tách khỏi relational database, phù hợp khi số nhân viên và số embedding tăng.
Quan sát vận hành	Backend expose /health và /metrics ; Prometheus scrape; Grafana hiển thị dashboard.	Báo cáo có bằng chứng runtime: service UP, queue length, tổng access logs và logs theo trạng thái.

Điểm thiết kế quan trọng

Chi tiết kỹ thuật không được nhồi hết vào sơ đồ để tránh rối. Sơ đồ chỉ thể hiện tầng và hướng phụ thuộc chính; phần bảng phía dưới giải thích data flow, ranh giới trách nhiệm và lý do chọn từng service. Đây là cách trình bày phù hợp hơn cho báo cáo hệ thống vì người đọc có thể hiểu kiến trúc trong 30 giây, sau đó đọc bảng để nắm chi tiết triển khai.

3.3 Vai trò từng service**Bảng 3.3:** Vai trò các service trong hệ thống

Service	Công nghệ	Vai trò
frontend-home	React/Vite/Nginx static	Trang vào hệ thống, điều hướng tới User Terminal hoặc Admin Console.
frontend-user	React/Vite	Giao diện gửi snapshot/webcam, xem trạng thái và lịch sử truy cập.
frontend-admin	React/Vite	Giao diện quản trị employees, users, cameras, enrollment và logs.
nginx	Nginx	Reverse proxy, gom UI và API về một entypoint <code>localhost:8080</code> .
backend	FastAPI/SQLAlchemy	Auth, role, CRUD, upload ảnh, tạo access log, queue job, health và metrics.

Service	Công nghệ	Vai trò
worker	Python/DeepFace	Lấy job từ Redis, tải ảnh từ MinIO, chạy DeepFace, search Qdrant và cập nhật PostgreSQL.
database	PostgreSQL 16	Source of truth cho users, employees, cameras, embeddings metadata và access logs.
redis	Redis 7	Queue cho <code>embedding_jobs</code> và <code>access_jobs</code> .
minio	MinIO	Object storage cho ảnh nhân viên, ảnh snapshot và backup.
qdrant	Qdrant	Vector database cho face embedding search.
prometheus	Prometheus	Scrape metrics từ backend và evaluate alert rules.
grafana	Grafana	Dashboard hiển thị health, queue length và access logs.
alertmanager	Alertmanager	Nhận cảnh báo từ Prometheus ở môi trường local.

3.4 Bằng chứng triển khai trong repo

Để báo cáo không chỉ dừng ở mức ý tưởng, bảng sau ánh xạ các khối kiến trúc sang thư mục hoặc tài liệu tương ứng trong repository.

Bảng 3.4: Mapping kiến trúc với artefact trong repo

Khối hệ thống	Artefact chính	Ý nghĩa khi bảo vệ
Backend API	backend/app	API nghiệp vụ, ORM model, service upload/storage, health và metrics.
Worker AI	worker/app	Xử lý Redis job, chạy DeepFace, search Qdrant và cập nhật PostgreSQL.
Queue	Redis + backend/app/queues	Hai queue riêng cho tạo embedding và check access.
Object storage	MinIO + storage service	Ảnh upload lưu bằng object key, không nhét file nhị phân vào PostgreSQL.
Vector search	Qdrant + vector service	Embedding được search bằng vector database, không so khớp tuần tự thủ công.
Monitoring	monitoring/	Dashboard, metrics và alert rules cho demo vận hành.
CI/CD và deploy	GitHub Actions + Helm	Kiểm thử/build tự động và baseline triển khai Kubernetes.

3.5 Lý do dùng xử lý bất đồng bộ

DeepFace là tác vụ nặng hơn nhiều so với CRUD thông thường. Nếu backend trực tiếp chạy phát hiện mặt và embedding trong request HTTP, API dễ bị chậm, timeout hoặc nghẽn

khi nhiều người gửi ảnh cùng lúc. Vì vậy backend chỉ nhận ảnh, lưu metadata, đẩy job vào Redis và trả trạng thái ban đầu; worker xử lý phía sau.

Lợi ích của queue

- ▶ API phản hồi nhanh, không bị khóa bởi model AI.
- ▶ Có thể scale worker độc lập nếu số lượng ảnh tăng.
- ▶ Dễ đo queue length để biết hệ thống đang backlog hay không.
- ▶ Có thể retry hoặc đánh dấu lỗi rõ ràng nếu pipeline AI thất bại.

Thiết kế dữ liệu

4.1 PostgreSQL là source of truth

PostgreSQL lưu dữ liệu nghiệp vụ chính. Các hệ thống khác như MinIO, Qdrant và Redis chỉ hỗ trợ lưu file, tìm kiếm vector hoặc queue job tạm thời.

Bảng 4.1: Các bảng dữ liệu chính

Bảng	Nội dung
users	Tài khoản đăng nhập, password hash, role <code>admin/user</code> và thời điểm tạo.
employees	Hồ sơ nhân viên, mã nhân viên, phòng ban, trạng thái <code>active/inactive</code> , trạng thái <code>embedding</code> .
cameras	Camera/cổng ra vào, vị trí, stream URL tùy chọn và trạng thái <code>active</code> .
face_embeddings	Vector embedding, model name, employee id và source image key.
access_logs	Lịch sử kiểm tra truy cập, status, score, camera, employee và message lỗi/kết quả.

Bảng 4.2: Các trường dữ liệu quan trọng cần nắm

Bảng	Trường chính	Ý nghĩa thiết kế
users	username, password_hash, role, is_active	Tách xác thực khỏi hồ sơ nhân viên; role quyết định quyền truy cập API.
employees	code, name, department, status, embedding_status, embedding_error	Lưu trạng thái nghiệp vụ và trạng thái AI riêng để admin biết employee đã sẵn sàng nhận diện hay chưa.
cameras	name, location, stream_url, is_active	Cho phép gắn access log với cổng/camera cụ thể, phục vụ audit sau này.
face_embeddings	employee_id, model_name, vector, source_image_key	PostgreSQL giữ metadata và vector gốc để có thể rebuild Qdrant nếu cần.
access_logs	employee_id, camera_id, status, score, image_path, message	Là audit trail của mỗi lần check access; <code>employee_id</code> có thể null nếu denied/error.

4.2 Quan hệ dữ liệu

Bảng 4.3: Quan hệ dữ liệu nghiệp vụ chính

Quan hệ	Kiểu	Ý nghĩa
employees → face_embeddings	1-n	Một nhân viên có thể có nhiều embedding từ nhiều ảnh.
employees → access_logs	1-n / nullable	Log có employee khi match; denied/error có thể null.
cameras → access_logs	1-n	Mỗi lần check access gắn với camera/cổng cụ thể.

4.3 MinIO, Qdrant và Redis

Bảng 4.4: Vai trò của các hệ thống lưu trữ hỗ trợ

Thành phần	Dữ liệu lưu	Lý do dùng
MinIO	Ảnh nhân viên, ảnh snapshot, backup PostgreSQL	File ảnh lớn không nên lưu trực tiếp trong PostgreSQL; object key được lưu trong DB để worker truy xuất.
Qdrant	Vector embedding và payload employee_id, model_name	Tìm kiếm vector gần nhất nhanh hơn truy vấn tuần tự trên database quan hệ.
Redis	Job tạm thời trong embedding_jobs, access_jobs	Tách backend request khỏi xử lý AI nền, đo backlog và scale worker.

Bảng 4.5: Quy ước object key và queue name

Loại dữ liệu	Quy ước	Ghi chú
Ảnh đăng ký nhân viên	employee-faces/...	Lưu ảnh nguồn dùng để tạo embedding; object key được ghi lại để debug và reindex.
Ảnh snapshot access	access-snapshots/...	Lưu ảnh mỗi lần người dùng check access; liên kết với access_logs.image_path.
Queue tạo embedding	embedding_jobs	Chỉ xử lý ảnh mẫu của employee.
Queue check access	access_jobs	Xử lý snapshot tại terminal và cập nhật access log.
Qdrant collection	deepface_embeddings	Payload gồm employee/model metadata để worker verify lại trong PostgreSQL.

4.4 Trạng thái quan trọng

Status cần nhớ

- ▶ Employee embedding_status: none, pending, success, error.
- ▶ Access log status: processing, granted, denied, error.
- ▶ Employee status: active hoặc inactive; employee inactive không được cấp quyền.

Bảng 4.6: Diễn giải trạng thái nghiệp vụ

Đối tượng	Trạng thái	Ý nghĩa
Employee embedding	none	Employee mới tạo, chưa upload ảnh hoặc chưa có job embedding.
Employee embedding	pending	Backend đã nhận ảnh và đẩy job vào Redis, worker chưa xử lý xong.
Employee embedding	success	Worker đã tạo embedding và upsert Qdrant thành công.
Employee embedding	error	Ảnh không hợp lệ, không detect được mặt, duplicate face hoặc lỗi dependency.
Access log	processing	Backend đã lưu snapshot và tạo job; kết quả cuối chưa có.
Access log	granted	Match employee active và score đạt threshold.
Access log	denied	Có embedding nhưng không vượt threshold hoặc không có candidate phù hợp.
Access log	error	Ảnh lỗi, không có mặt, nhiều mặt hoặc worker gặp lỗi khi xử lý.

Pipeline AI nhận diện khuôn mặt

5.1 Cấu hình AI

Worker sử dụng DeepFace để phát hiện khuôn mặt và tạo embedding. Cấu hình mặc định trong repo:

Bảng 5.1: Cấu hình AI mặc định

Biến / thành phần	Giá trị	Ý nghĩa
DEEPFACE_MODEL_NAME	Facenet512	Model tạo embedding khuôn mặt.
DEEPFACE_DETECTOR_BACKEND	mtcnn	Detector dùng chung cho cả đăng ký khuôn mặt và check access.
DEEPFACE_MATCH_THRESHOLD	0.70	Ngưỡng cấp quyền khi score đủ cao.
DEEPFACE_DUPLICATE_THRESHOLD	0.95	Ngưỡng chặn đăng ký trùng khuôn mặt giữa hai employee.
QDRANT_COLLECTION	deepface_embeddings	Collection lưu vector face embedding.

5.2 Luồng đăng ký khuôn mặt

Enrollment là quá trình admin upload ảnh mẫu để hệ thống tạo embedding cho nhân viên.

Flow enrollment

Admin upload ảnh → Backend validate → MinIO lưu object key → Redis `embedding_jobs` → Worker detect face → DeepFace tạo embedding → Qdrant duplicate check → PostgreSQL + Qdrant lưu kết quả.

1. Admin tạo employee và upload ảnh khuôn mặt từ Admin Console.
2. Backend kiểm tra định dạng, content type, dung lượng và employee id.
3. Ảnh được lưu vào MinIO với prefix `employee-faces/{employee_id}`.
4. Backend đẩy job vào Redis queue `embedding_jobs`.
5. Worker tải ảnh từ MinIO về file tạm và yêu cầu đúng một khuôn mặt.
6. DeepFace tạo vector embedding bằng model Facenet512.
7. Worker kiểm tra trùng khuôn mặt trong Qdrant để tránh đăng ký một người cho nhiều employee.
8. Vector được lưu vào PostgreSQL và upsert vào Qdrant; employee chuyển sang `embedding_status=success`.

Payload embedding job

Khi backend queue job đăng ký khuôn mặt, worker chỉ cần biết loại job, employee id và object key ảnh. Ví dụ logic:

```
{
  "type": "employee_embedding",
  "employee_id": 8,
  "image_key": "employee-faces/8/4b6f...jpg"
}
```

Backend không truyền ảnh nhị phân qua Redis. Redis chỉ giữ metadata nhỏ; file thật nằm trong MinIO. Thiết kế này giúp queue nhẹ và tránh mất dữ liệu ảnh khi job chưa được xử lý.

Bảng 5.2: Kiểm soát chất lượng trong enrollment

Bước kiểm soát	Điều kiện	Kết quả nếu không đạt
Validate upload	File ảnh hợp lệ, đúng content type, không vượt giới hạn dung lượng.	API trả lỗi, không lưu vào MinIO và không queue job.
Detect face	Ảnh phải có đúng một khuôn mặt.	Employee chuyển <code>embedding_status=error</code> .
Duplicate check	Vector không quá giống employee active khác theo <code>DEEPFACE_DUPLICATE_THRESHOLD</code> .	Worker báo duplicate face để tránh một người gắn vào nhiều hồ sơ.
Upsert vector	PostgreSQL và Qdrant đều cập nhật thành công.	Ghi <code>embedding_error</code> để admin biết cần upload lại hoặc kiểm tra service.

5.3 Luồng kiểm tra ra vào

Access check là luồng người dùng gửi ảnh snapshot để xác minh quyền ra vào.

Flow access check

User gửi snapshot → Backend upload MinIO → tạo access log `processing` → Redis `access_jobs` → Worker detect + embedding → Qdrant search → PostgreSQL verify → cập nhật `granted`, `denied` hoặc `error`.

Khi backend trả về `processing`, điều đó không có nghĩa là hệ thống đã nhận diện xong. Nó chỉ có nghĩa request đã được nhận, ảnh đã lưu và job đã vào queue. Kết quả cuối cùng nằm trong bảng `access_logs` sau khi worker cập nhật.

Payload access job

Access job chứa id của access log để worker cập nhật đúng record sau khi xử lý:

```
{
  "type": "access_check",
  "access_log_id": 125,
  "camera_id": 1,
  "image_key": "access-snapshots/9c2a...jpg"
}
```

Nhờ có `access_log_id`, UI có thể hiển thị trạng thái **processing** ngay lập tức và polling/log refresh để nhận kết quả cuối.

Bảng 5.3: Quy tắc ra quyết định access

Tình huống	Status cuối	Giải thích
Không đọc được ảnh hoặc MinIO lỗi	error	Worker không có input hợp lệ để chạy model.
Không detect được mặt hoặc có nhiều mặt	error	Hệ thống yêu cầu mỗi request là một người để tránh quyết định sai.
Qdrant không trả candidate phù hợp	denied	Ảnh hợp lệ nhưng không giống employee đã đăng ký.
Candidate có score dưới threshold	denied	Có khuôn mặt gần nhất nhưng độ tin cậy chưa đủ để mở cổng.
Employee match nhưng inactive	denied	PostgreSQL là source of truth, trạng thái nhân viên quyết định quyền cuối.
Employee active và score đạt threshold	granted	Hệ thống ghi employee id, score và message vào access log.

5.4 Quy tắc xử lý lỗi

Pipeline AI yêu cầu mỗi ảnh chỉ có đúng một khuôn mặt. Nếu không có mặt, có nhiều mặt, ảnh lỗi hoặc dependency MinIO/Qdrant/PostgreSQL gặp lỗi, access log sẽ chuyển sang **error**. Nếu có mặt nhưng score không đủ threshold, log là **denied**. Nếu match với employee active và score vượt threshold, log là **granted**.

Lưu ý về threshold

Ngưỡng `DEEPFACE_MATCH_THRESHOLD=0.70` phù hợp cho demo, nhưng không nên xem là hằng số production. Camera, ánh sáng, góc mặt, khoảng cách, kính, khẩu trang và chất lượng ảnh đều ảnh hưởng đến score. Khi triển khai thật cần thu thập dữ liệu tại hiện trường và hiệu chỉnh ngưỡng theo false accept rate và false reject rate.

Backend API

6.1 Nhóm endpoint chính

Bảng 6.1: Nhóm API backend

Nhóm API	Endpoint tiêu biểu	Vai trò
Auth	POST /auth/login, GET /auth/me	Đăng nhập, cấp Bearer token, lấy thông tin user hiện tại.
Employees	GET/POST/PUT/DELETE /employees	Quản lý hồ sơ nhân viên.
Face image	POST /employees/{id}/face-image	Upload ảnh nhân viên và queue job tạo embedding.
Cameras	GET /cameras, GET /cameras/active-default	Lấy camera/cổng mặc định cho User UI.
Access	POST /access/check-image	Upload snapshot và queue job kiểm tra quyền.
Logs	GET /logs	Xem lịch sử truy cập.
Admin	GET /admin/status, GET /admin/users	Trạng thái vận hành và quản lý user.
Operations	GET /health, GET /metrics	Health check và Prometheus metrics.

Bảng 6.2: API contract quan trọng

Endpoint	Input chính	Output / side effect
POST /auth/login	Username/password	Trả Bearer token; frontend lưu token để gọi API tiếp theo.
GET /auth/me	Bearer token	Trả thông tin user hiện tại và role để UI điều hướng quyền.
POST /employees	Code, name, department, status	Tạo employee; mặc định chưa có embedding sẵn sàng.
POST /employees/{id}/face-image	Multipart image	Upload ảnh vào MinIO, set <code>embedding_status=pending</code> , queue <code>embedding_jobs</code> .
GET /employees	Filter/pagination nếu có	Trả danh sách nhân viên kèm trạng thái embedding để admin theo dõi.
POST /access/check-image	Multipart image, camera id	Tạo <code>access_logs.status=processing</code> , upload snapshot, queue <code>access_jobs</code> .
GET /logs	Token, query filter	Trả lịch sử access: status, employee, camera, score, message, thời gian.
GET /health	Không cần body	Trả trạng thái backend, database và Redis để kiểm tra readiness.

Endpoint	Input chính	Output / side effect
GET /metrics	Không cần body	Trả Prometheus metrics cho dashboard và alert rules.

Bảng 6.3: HTTP status code cần giải thích khi bảo vệ

Code	Tình huống	Ý nghĩa
200/201	Login, CRUD, query thành công	Request hợp lệ và backend xử lý xong phần nghiệp vụ tức thời.
202 hoặc response processing	Access/enrollment đã queue	AI chưa xử lý xong; frontend cần đọc log/status sau.
400	Upload sai định dạng, file rỗng, file quá lớn	Backend chặn input xấu trước khi vào MinIO/worker.
401	Thiếu hoặc sai token	Người gọi chưa xác thực.
403	Sai role	User thường không được gọi API admin.
404	Employee/camera/log không tồn tại	Resource id không hợp lệ hoặc đã bị vô hiệu hóa.
500	Lỗi dependency hoặc lỗi server	Cần xem backend/worker logs và metrics để truy nguyên.

6.2 Auth và phân quyền

Backend sử dụng Bearer token. Có hai vai trò chính:

Bảng 6.4: Vai trò người dùng

Role	Quyền
admin	Quản lý employee, user, camera, xem log, upload ảnh mẫu và xem trạng thái hệ thống.
user	Dùng User Terminal để gửi ảnh check access và xem thông tin cần thiết cho terminal.

6.3 Validate upload ảnh

Các endpoint upload ảnh áp dụng rule chung: file không rỗng, extension hợp lệ, content type hợp lệ và dung lượng không vượt quá 5 MB. Quy tắc này giúp backend chặn file rác trước khi đưa vào MinIO và trước khi worker phải xử lý.

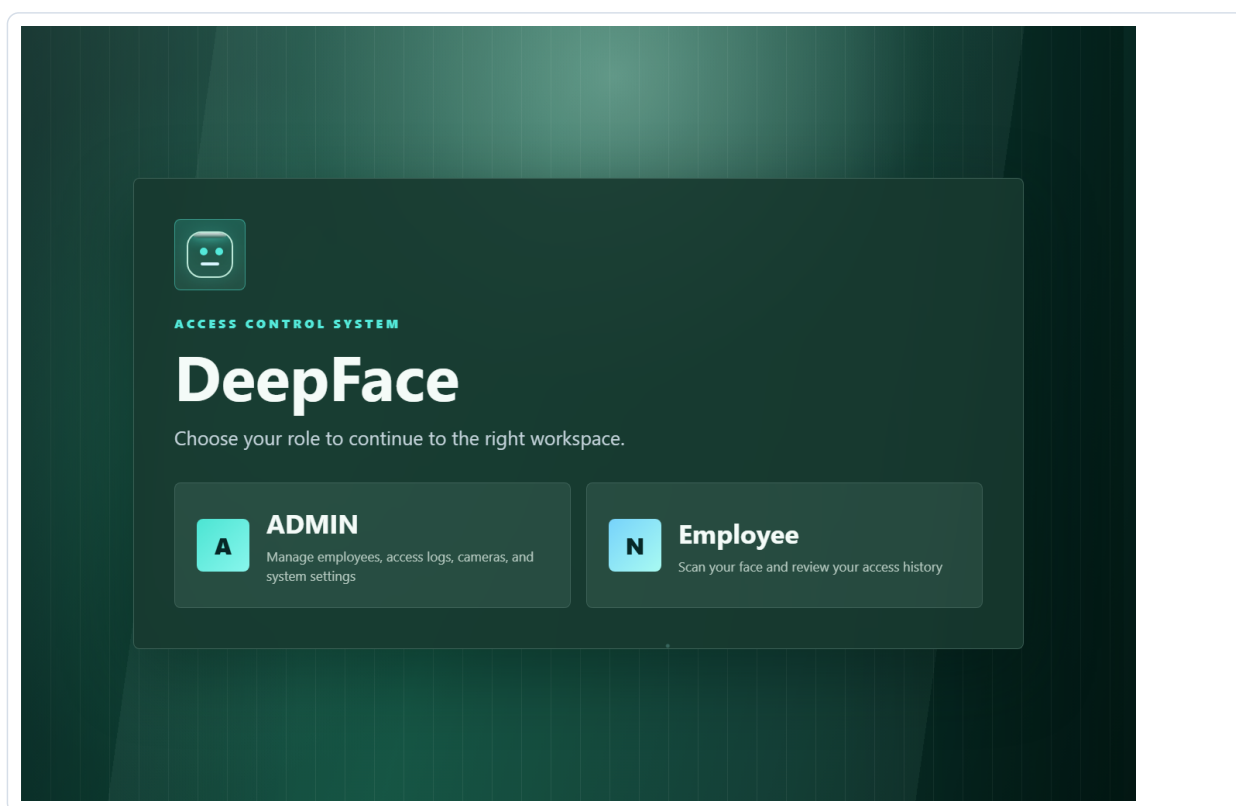
Vì sao không lưu ảnh vào PostgreSQL

Ảnh là object nhị phân lớn, không phù hợp để lưu trực tiếp trong bảng nghiệp vụ. MinIO lưu file thật; PostgreSQL chỉ lưu object key như `employee-faces/...` hoặc `access-snapshots/...`. Cách này rõ trách nhiệm, dễ backup/migrate và giống kiến trúc production hơn.

Thiết kế giao diện

7.1 Home Gateway

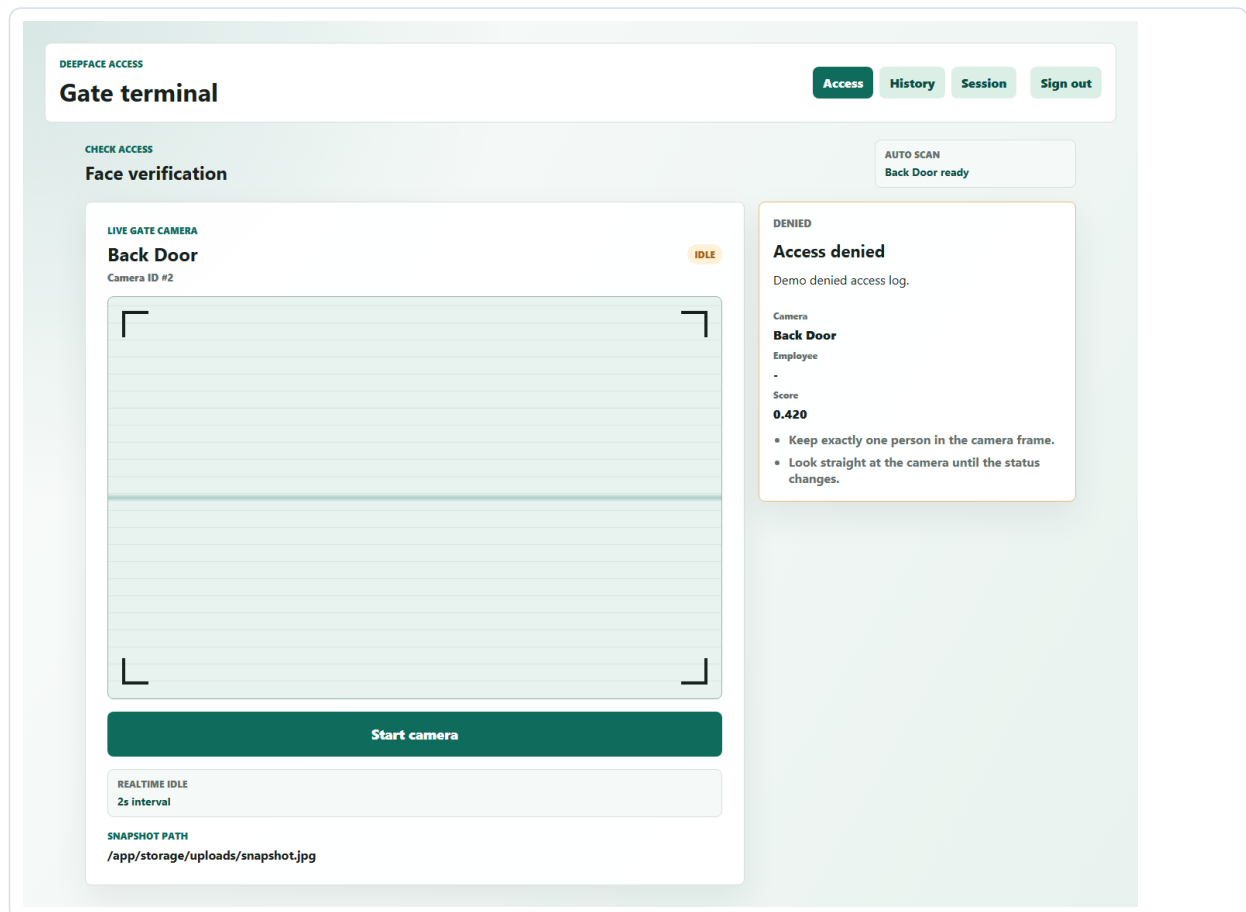
Home UI là điểm vào đầu tiên, giúp người dùng chọn đúng khu vực làm việc. Việc tách Home, User và Admin giúp demo rõ ràng hơn: người dùng terminal không nhìn thấy chức năng quản trị; admin có dashboard riêng để cấu hình hệ thống.



Hình 7.1: Giao diện Home Gateway

7.2 User Terminal

User Terminal tập trung vào tác vụ chính: gửi ảnh để kiểm tra quyền ra vào. Giao diện cần ít nhiều, trạng thái dễ đọc và hiển thị lịch sử kết quả gần nhất.



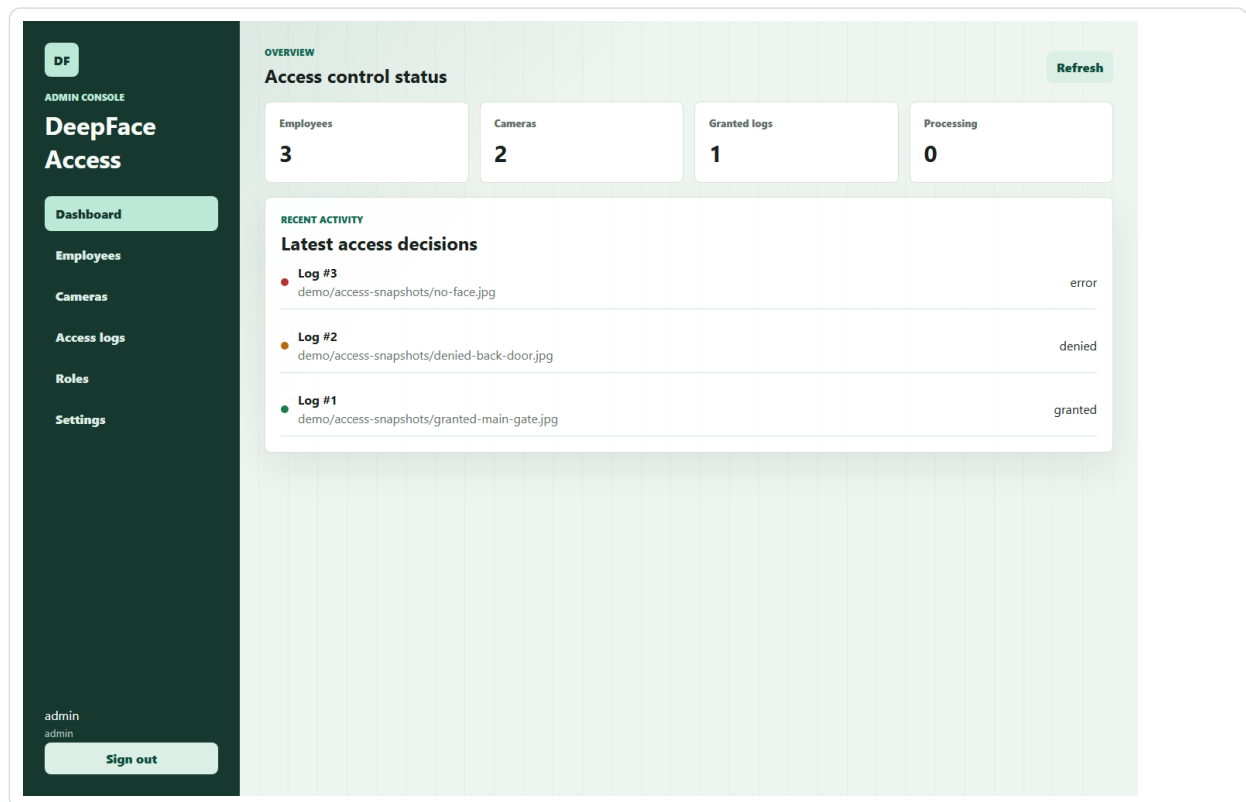
Hình 7.2: Giao diện User Terminal

Chức năng chính của User Terminal:

- ▶ Đăng nhập bằng tài khoản user.
- ▶ Lấy camera active/default từ backend.
- ▶ Upload snapshot hoặc chụp frame từ webcam.
- ▶ Gửi request `/access/check-image`.
- ▶ Theo dõi trạng thái `processing`, `granted`, `denied`, `error`.

7.3 Admin Console

Admin Console phục vụ các tác vụ quản trị: tạo nhân viên, upload ảnh mẫu, theo dõi embedding status, quản lý user, xem access logs và kiểm tra trạng thái hệ thống.



Hình 7.3: Giao diện Admin Console

Chức năng chính của Admin Console:

- ▶ Dashboard thống kê employees, cameras, users và logs.
- ▶ Tạo, cập nhật và vô hiệu hóa nhân viên.
- ▶ Upload ảnh khuôn mặt cho enrollment.
- ▶ Theo dõi embedding status và lỗi nếu có.
- ▶ Quản lý user và xem lịch sử access.

Monitoring và vận hành

8.1 Mục tiêu monitoring

Monitoring giúp chứng minh hệ thống không chỉ chạy được chức năng chính mà còn có khả năng quan sát vận hành. Backend cung cấp `/health` để kiểm tra database/Redis và `/metrics` cho Prometheus.

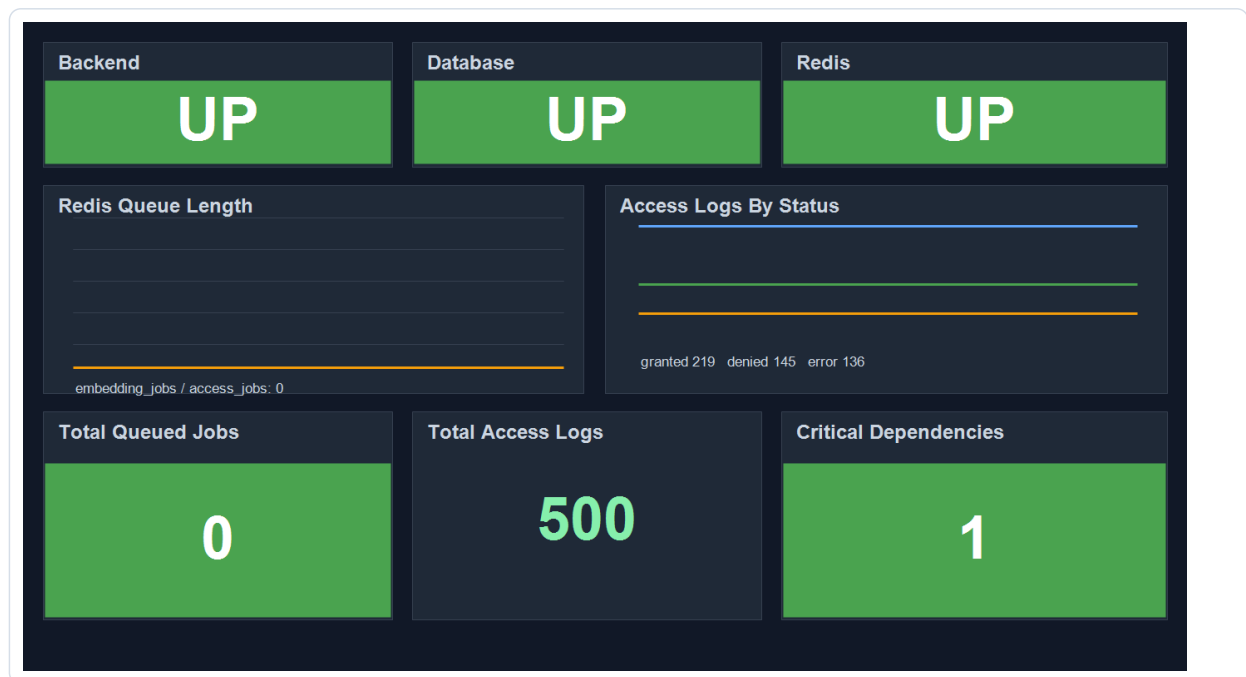
Bảng 8.1: Metric chính

Metric	Ý nghĩa
<code>deepface_backend_up</code>	Backend process còn hoạt động hay không.
<code>deepface_database_up</code>	Backend kết nối được PostgreSQL hay không.
<code>deepface_redis_up</code>	Backend kết nối được Redis hay không.
<code>deepface_queue_length</code>	Số job đang chờ trong từng queue.
<code>deepface_access_logs_total</code>	Số access logs theo trạng thái.

Bảng 8.2: Cách đọc metric khi demo

Metric / panel	Giá trị tốt	Ý nghĩa vận hành
Backend/Database/Redis UP	1 hoặc UP	Ba dependency tối thiểu cho API và queue đang sẵn sàng.
<code>deepface_queue_length</code> {queue="access_jobs"}	Gần 0 khi idle	Nếu tăng liên tục nghĩa là worker không xử lý kịp hoặc đang lỗi.
<code>deepface_queue_length</code> {queue="embedding_jobs"}	Gần 0 khi idle	Nếu còn backlog, employee mới upload chưa có embedding sẵn sàng.
<code>deepface_access_logs_total</code> {status="processing"}	Không tăng lâu	Nhiều log processing kéo dài cho thấy job bị kẹt.
<code>deepface_access_logs_total</code> {status="error"}	Không tăng bất thường	Error tăng có thể do ảnh lỗi, model không detect được mặt hoặc dependency hỏng.

8.2 Grafana dashboard



Hình 8.1: Dashboard Grafana DeepFace Access Overview

Dashboard Grafana hiển thị các thông tin quan trọng cho demo:

- ▶ Backend, Database và Redis đang UP.
- ▶ Redis queue length cho `embedding_jobs` và `access_jobs`.
- ▶ Access logs theo trạng thái `granted`, `denied`, `error`.
- ▶ Total queued jobs, total access logs và critical dependencies.

8.3 Alertmanager

Prometheus có rule cảnh báo khi backend metrics down, database/Redis unavailable, queue backlog cao, nhiều access log kẹt `processing` hoặc có log `error`. Alertmanager hiện dùng local receiver, phù hợp cho demo; khi production có thể thêm Slack, email hoặc webhook.

Bảng 8.3: Alert rule và hướng xử lý

Alert	Điều kiện	Cách xử lý khi demo
RedisUnavailable	<code>deepface_redis_up == 0</code>	Kiểm tra container Redis, backend env <code>REDIS_URL</code> , sau đó restart backend/worker nếu cần.
QueueBacklogHigh	Tổng queue length vượt ngưỡng	Xem <code>docker compose logs worker</code> ; có thể worker đang load model hoặc lỗi MinIO/Qdrant.
AccessProcessingBacklog	Nhiều log bị processing lâu	Kiểm tra access job có được worker consume hay không.
AccessErrorsPresent	Có log <code>error</code>	Mở Admin Console hoặc logs API để xem message; nếu do ảnh no-face thì đây là lỗi dữ liệu đầu vào, không phải lỗi hệ thống.

Triển khai Docker, CI/CD và backup

9.1 Docker Compose

Docker Compose gom toàn bộ hệ thống vào một môi trường local. Người chấm chỉ cần chuẩn bị Docker Desktop, copy file môi trường và chạy compose.

```
Copy-Item .env.example .env
docker compose up -build -d
```

Các URL chính:

Bảng 9.1: Endpoint truy cập sau khi chạy Docker Compose

Thành phần	URL
Home Gateway	http://localhost:8080
User Terminal	http://localhost:8080/user/
Admin Console	http://localhost:8080/admin/
Backend Swagger	http://localhost:8080/docs
MinIO Console	http://localhost:9001
Qdrant REST	http://localhost:6333
Prometheus	http://localhost:9090
Grafana	http://localhost:3000

Bảng 9.2: Nhóm biến môi trường quan trọng

Nhóm	Biến tiêu biểu	Vai trò
Database/queue	DATABASE_URL, REDIS_URL	Backend và worker dùng để đọc ghi nghiệp vụ và consume queue.
Object storage	MINIO_ENDPOINT, MINIO_ROOT_USER, MINIO_ROOT_PASSWORD, MINIO_BUCKET	Cấu hình nơi lưu ảnh employee/access snapshot.
Vector database	QDRANT_URL, QDRANT_COLLECTION	Cấu hình index vector và endpoint search.
Auth	AUTH_SECRET_KEY, token expiry nếu có	Ký Bearer token và kiểm soát session.
AI	DEEPFACE_MODEL_NAME, DEEPFACE_DETECTOR_BACKEND, DEEPFACE_MATCH_THRESHOLD	Chọn model, detector và ngưỡng nhận diện.
Frontend/API	API base URL, CORS origins	Cho phép UI gọi đúng backend qua Nginx trong local demo.

9.2 CI/CD baseline

GitHub Actions trong repo thực hiện:

- ▶ Backend tests bằng Python 3.12.
- ▶ Worker tests với dependency OpenCV/DeepFace cần thiết.
- ▶ Frontend builds cho User/Admin.
- ▶ Docker image builds cho backend, worker, frontend-home, frontend-user và frontend-admin.
- ▶ Publish image lên Docker Hub khi push vào `main` nếu cấu hình secret.

Phạm vi CI/CD hiện tại

Pipeline hiện là CI + Docker image publishing. Điều này đủ để chứng minh repo có kiểm thử/build tự động và sinh artifact deploy được. Auto-deploy lên VPS hoặc Kubernetes bằng Helm là hướng mở rộng tiếp theo.

9.3 Helm baseline

Repo có Helm chart tại `helm/deepface-access`. Chart này là nền tảng triển khai Kubernetes cho backend, worker, frontend, database, Redis, MinIO, Qdrant và ingress. Với môi trường production, cần cấu hình image registry, image tag, resource request/limit, Secret, persistent volume và TLS/Ingress.

9.4 Backup

Repo có ba hướng backup:

Bảng 9.3: Chiến lược backup

Cách backup	File/service	Mục đích
Backup local	<code>scripts/backup.ps1</code>	Dump PostgreSQL ra thư mục backup local.
Backup S3 từ host	<code>scripts/backup-s3.ps1</code>	Dump PostgreSQL rồi upload lên S3/MinIO/S3-compatible storage.
Backup tự động	<code>cron-backup</code>	Container chạy cron, dump PostgreSQL định kỳ và upload vào MinIO/S3.

Trong phạm vi MVP, backup tập trung vào PostgreSQL vì đây là dữ liệu nghiệp vụ chính. MinIO bucket ảnh và Qdrant snapshot nên được bổ sung vào chiến lược backup production.

Kiểm thử và đánh giá

10.1 Chiến lược kiểm thử

Bảng 10.1: Nhóm kiểm thử trong repo

Nhóm kiểm thử	Nội dung	Lệnh
Backend API	Auth, employees, cameras, access, logs, admin, health, metrics.	<code>pytest backend/app/tests</code>
Worker services	Detector, embedder, vector store, queue handlers và job processors.	<code>pytest worker/app/tests</code>
Frontend build	TypeScript compile và Vite production build.	<code>npm run build</code>
Runtime baseline	Health, metrics, frontend routes, monitoring và queue status.	<code>demo-baseline-check.ps1</code>
AI smoke test	Tạo embedding và check granted/denied/error với ảnh mẫu.	<code>smoke-deepface.ps1</code>

Bảng 10.2: Acceptance criteria cho demo bảo vệ

Kịch bản	Kết quả đạt	Bằng chứng nên mở khi trình bày
Stack khởi động	Các service chính Up/healthy; Nginx mở được localhost:8080.	<code>docker compose ps</code> , Home Gateway.
Admin tạo employee	Employee xuất hiện trong Admin Console, status active.	Employee table hoặc API response.
Upload ảnh employee	<code>embedding_status</code> từ <code>pending</code> sang <code>success</code> .	Admin Console, MinIO object, worker logs, Qdrant collection.
User check đúng người	Access log từ <code>processing</code> sang <code>granted</code> , có employee và score.	User Terminal, Admin logs, PostgreSQL log nếu cần.
User check sai người	Access log <code>denied</code> .	User/Admin logs và message.
Ảnh lỗi/no-face	Access log <code>error</code> kèm message rõ.	Admin logs và worker logs.
Monitoring	Dashboard hiển thị Backend/Database/Redis UP, queue length, access logs by status.	Grafana dashboard trong Hình 8.1.

10.2 Đánh giá MVP

MVP đạt mục tiêu chứng minh luồng end-to-end: admin đăng ký nhân viên, upload ảnh, worker tạo embedding, user gửi snapshot, worker match Qdrant và hệ thống ghi access

log. Điểm mạnh của repo là kiến trúc nhiều service, có queue nền, object storage, vector database, monitoring, CI/CD baseline và tài liệu kỹ thuật theo từng mảng.

Bảo mật và rủi ro

11.1 Bảo mật hiện tại

Hệ thống đã có các lớp bảo mật cơ bản phù hợp với MVP:

- ▶ Bearer token cho API.
- ▶ Password được hash, không trả về password hash qua API.
- ▶ Role admin/user để giới hạn API quản trị.
- ▶ CORS cấu hình qua biến môi trường.
- ▶ File môi trường thật không nên commit; dùng `.env.example`.
- ▶ Upload ảnh có validate extension, content type và size.

11.2 Rủi ro cần hardening

Các điểm cần xử lý trước production

- ▶ Đổi toàn bộ credential demo như `admin/admin123`, `minioadmin/minioadmin`.
- ▶ Bật HTTPS khi expose ra Internet.
- ▶ Siết quyền đọc access logs để user không xem dữ liệu ngoài phạm vi.
- ▶ Dùng Secret Manager hoặc Kubernetes Secret cho credential nhạy cảm.
- ▶ Thêm audit log cho thao tác admin như tạo user, xóa employee, đổi role.
- ▶ Thêm rate limit để tránh spam upload hoặc brute force login.

11.3 Dữ liệu sinh trắc học

Khuôn mặt là dữ liệu sinh trắc học nhạy cảm. Khi triển khai thực tế cần thông báo rõ mục đích thu thập, phạm vi sử dụng, thời gian lưu trữ và quy trình xóa dữ liệu. Ảnh gốc trong MinIO cần giới hạn quyền truy cập, backup cần có retention và không nên chia sẻ bucket công khai.

Phân công nhóm và kế hoạch bảo vệ

12.1 Phân công nhóm 4 người

Bảng 12.1: Gợi ý phân công theo module

Thành viên	Phụ trách	Nội dung chính
Thành viên 1	Backend/API	Auth, role, employees, access, logs, upload, health/metrics, database schema.
Thành viên 2	AI/Worker	DeepFace, detector, embedding, Qdrant search, threshold, queue jobs, xử lý granted/denied/error.
Thành viên 3	Frontend	Home, User Terminal, Admin Console, webcam/snapshot, trạng thái UI, lỗi người dùng.
Thành viên 4	DevOps/Docs/QA	Docker Compose, Nginx, MinIO, Redis, monitoring, CI/CD, backup, README, demo checklist.

12.2 Kịch bản demo đề xuất

1. Chạy `docker compose up -build -d`.
2. Mở Home Gateway tại <http://localhost:8080>.
3. Đăng nhập Admin, tạo employee và upload ảnh khuôn mặt.
4. Chờ `embedding_status=success`.
5. Đăng nhập User Terminal, gửi snapshot của người đã đăng ký.
6. Xem log chuyển từ `processing` sang `granted`.
7. Gửi ảnh sai người/no-face để minh họa `denied/error`.
8. Mở Grafana để chứng minh backend, database, Redis, queue và access logs được quan sát.

Kết luận và hướng phát triển

13.1 Kết luận

Đồ án đã xây dựng được một MVP kiểm soát ra vào bằng nhận diện khuôn mặt với đầy đủ thành phần cốt lõi của một hệ thống AI thực tế: frontend cho user/admin, backend API, worker AI, database, object storage, vector database, queue, gateway và monitoring. Cách tổ chức theo service giúp hệ thống rõ trách nhiệm, dễ demo, dễ mở rộng và phù hợp với yêu cầu môn học.

Hệ thống chứng minh được luồng nghiệp vụ chính: đăng ký nhân viên, upload ảnh mẫu, tạo embedding, gửi snapshot kiểm tra quyền, so khớp vector và ghi access log. Đây là nền tảng đủ tốt để tiếp tục mở rộng sang đánh giá accuracy, hardening bảo mật và triển khai staging/production.

13.2 Hướng phát triển

1. Thêm Alembic migrations để version database schema rõ ràng.
2. Bổ sung Playwright E2E tests cho luồng Admin và User.
3. Hỗ trợ nhiều ảnh/embedding cho mỗi employee ở nhiều góc mặt khác nhau.
4. Bổ sung reindex job cho Qdrant khi đổi model hoặc rebuild collection.
5. Thêm worker metrics và latency histogram cho access pipeline.
6. Mirror MinIO bucket và snapshot Qdrant trong chiến lược backup.
7. Hoàn thiện Helm values, Secret và Ingress/TLS cho Kubernetes.

13.3 Đánh giá tổng thể

Nhận định cuối

MVP đáp ứng đúng yêu cầu bài tập lớn ở mức hệ thống: có use case rõ, có frontend user/admin, backend, database, object storage, vector database, queue/worker, Nginx, Docker Compose và monitoring. Phần nên đầu tư tiếp theo không phải thêm nhiều màn hình, mà là đo accuracy có kiểm soát, hardening bảo mật và tự động hóa kiểm thử end-to-end.

Tài liệu tham khảo

1. Serengil, S. I., & Ozpinar, A. (2020). LightFace: A Hybrid Deep Face Recognition Framework.
2. DeepFace GitHub Repository: <https://github.com/serengil/deepface>
3. FastAPI Documentation: <https://fastapi.tiangolo.com/>
4. Qdrant Documentation: <https://qdrant.tech/documentation/>
5. MinIO Documentation: <https://min.io/docs/>
6. Docker Compose Documentation: <https://docs.docker.com/compose/>
7. Prometheus Documentation: <https://prometheus.io/docs/>
8. Grafana Documentation: <https://grafana.com/docs/>